From Graphs to Transformers, a Review on Modern Architectures and Implementations for Atomistic Machine Learning

Aryan Saha
as7959@princeton.edu

February 2026

This paper is a work-in-progress and contains errors. Please suggest possible corrections to the authors.

Abstract

Computational models applied to 3D atomistic structures have undergone tremendous advancement and interest in regards to accelerating scientific research and analysis. The state-of-the-art models utilize graph neural network architectures with an emphasis on utilizing physical inductive biases. However, similarities of GNNs with more parallelizable and scalable architectures, such as Transformers, have spiked interest in whether such models may benefit from other methods. This review paper intends to construct an intuition behind inducing geometric priors, the connection between graph neural networks and Transformers, and then provide a guide towards the development and possible use of attention-based architectures with induced geometric priors.

1 Introduction

Within the field of computational analysis and research of materials, molecules, and general atomistic systems, machine learning models have demonstrated great potential to accelerate atomistic calculations. The current state of computational chemistry relies on *ab initio* calculation methods via Density Functional Theory (DFT), which offers varying levels of approximations of energies, forces, and other quantum-chemical mechanical properties.

Advancements in machine learning and artificial intelligence have demonstrated great promise in accelerating the process of obtaining these calculations. DFT requires high computational power and is typically slow. However, many researchers have collected DFT calculations for many sample materials, such as datasets of MP, QMOF, QM9, and more. The existence of these datasets allows us to try to use methods in AI/ML and try to accelerate the calculations.

MLIPs and most molecular machine learning models utilize a graph neural network architecture, where nodes resemble atoms and bonds resemble edges. This architecture has demonstrated superior performance in recent benchmarks (cite models and matbench discovery). State of the art GNN-based MLIPs incorporate strong physical inductive biases within the model's architecture design. However, we acknowledge the connection between GNN architectures and popular Transformer architectures [Joshi, 2025]. Based on the interest behind transformer architectures, such as their ability for parallelization and avoid sparse computations, we seek to illustrate an intuition on how to induce equivariance in Transformer architectures.

1.1 Primer on Group Theory

As much of this paper will focus on methods of inducing physical bias in machine learning interatomic potential architectures, we must outline the definition of one of the most fundamental and commonly implemented biases, equivariance, and invariance. First, we define a group $\mathfrak{G}$ as a set. We also define a binary operator $\circ : \mathfrak{G} \circ \mathfrak{G} \to \mathfrak{G}$. A group obeys these axioms:

1. Identity: There should exist an element $c \in \mathfrak{G}$ such that $c \circ g = g \circ c = g$ for all $g \in \mathfrak{G}$

2. Associativity: $(a \circ b) \circ c = a \circ (b \circ c)$ for all $a, b, c \in \mathfrak{G}$.

3. Inverse: $\forall g \in \mathfrak{G}$, there exists a unique $g^{-1} \in \mathfrak{G}$ such that $g^{-1} \circ g = g \circ g^{-1} = e$

4. Closure: For all $c, d \in \mathfrak{G}$, $c \circ d \in \mathfrak{G}$

Now that we have defined groups and group theory, we can introduce common Lie groups which are utilized in geometric deep learning:

1. E(3), Euclidean Group in 3-Dimensions: 3D rotations, translations, inversions

2. SE(3), Special Euclidean Group in 3-Dimensions: 3D rotations, translations

3. O(3), Orthogonal Group in 3-Dimensions: 3D rotations, inversions

4. SO(3), Special Orthogonal Group in 3-Dimensions: 3D rotations

For the $E(3)$ and $SE(3)$ Lie groups, the elements are represented as pairs, (R, t), given that $R \in \mathbb{R}^{3\times3}$, $t \in \mathbb{R}^3$ and $R^\top R = I$, with $\det R = 1$ for SE(3). For the orthogonal groups O(3) and SO(3), elements are represented as matrices $\mathrm{R} \in \mathbb{R}^{3\times3}$, where R is an orthogonal matrix, with $\det R = 1$ in the case of SO(3). Having defined these special groups, we can interpret equivariance/invariance in respect to various Lie groups.

Equivariance: Equivariance can be defined as the property that when a function's input is subjected to a transformation, the output is transformed in a corresponding manner.

$$f(D_x(g)\,\vec{x}) = D_y(g)\,f(\vec{x}) \quad \forall g \in \mathfrak{G} \tag{1}$$

In (1), we define the function $f : \mathcal{X} \to \mathcal{Y}$ as equivariant given: state input space $\mathcal{X}$ and output space $\mathcal{Y}$, group representations $D_x(g)$ for input and $D_y(g)$ for output, Lie group $\mathfrak{G}$, and input vector $\vec{x}$.

Invariance: Invariance can be defined as the property that when a function's input is subject to a transformation, the output remains unchanged.

$$f(D_x(g)\,\vec{x}) = f(\vec{x}) \quad \forall g \in \mathfrak{G} \tag{2}$$

In (2), we define the function $f : \mathcal{X} \to \mathcal{Y}$ as invariant given input space $\mathcal{X}$ and output space $\mathcal{Y}$, group representations $D_x(g)$ for inputs and $D_y(g) = I$ for outputs, Lie group $\mathfrak{G}$, and input vector $\vec{x}$.

Having defined equivariance and invariance with respect to Lie groups, we can investigate methods of implementing equivariance/invariance in atomistic machine learning models.

2 Geometric Priors for Graph Neural Networks

Graph Neural Networks (GNNs) are the most popular architecture used for molecular and material machine learning models. In GNNs, nodes represent atoms and edges represent bonds. Using a formal notation, we let the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where set $\mathcal{V}$ represents nodes and set $\mathcal{E}$ represents edges. The adjacency matrix $A \in \mathcal{R}^{N\times N}$ includes entries a_{ij} which indicate the presence of an edge between nodes i and j. Nodes may also contain features, introducing an input matrix $\mathcal{X} \in \mathcal{R}^{N\times F}$, with hidden node embeddings at layer L being $\mathcal{H}^{(l)}$ where $\mathcal{X} = \mathcal{H}^{(0)}$. Within the application space of chemistry and material science, these graphs are undirected and possibly cyclic, resulting in symmetric adjacency matrices.

GNNs can be defined via *message passing*, where layers are updated via the passing of message vectors. Edge vectors are first updated, then node vectors, and finally global values. [Gilmer et al., 2017, Battaglia et al., 2018]:

$$h_u^{(l+1)} = \phi\left(\mathbf{h_u}^{(l)}, \bigoplus_{v\in\mathcal{N}_u} \psi(\mathbf{h_u}^{(l)}, \mathbf{h_v}^{(l)}, \mathbf{e_{uv}^{(l)}})\right) \tag{3}$$

where:

$$m_{uv}^{(l)} = \psi(\mathbf{h_u}^{(l)}, \mathbf{h_v}^{(l)}, \mathbf{e_{uv}^{(l)}}) \tag{4}$$

In Eq. (20), we first construct a learnable message function ψ between nodes u and v via node embeddings ${h_u}^{(l)}, {h_v}^{(l)}$ and edge $e_{uv}^{(l)}$. We then use a permutation-invariant operator $\bigoplus$, commonly summation or multiplication. Then, through an additional learnable function ϕ, we update u using this result. We update node u using all edges, such that $v \in \mathcal{N}_u$, where $\mathcal{N}_u$ are all neighboring nodes of node u. Learnable message functions ψ and ϕ can take the form of affine functions with activations for multi-perceptron layers. From this general formula, we demonstrate the basic functionalities of all GNNs. With additional layers, information from nodes can traverse throughout the graph, numerous neighbors away.

2.1 Methods of Geometric Constraints: Equivariance and Invariance

Vanilla Graph Neural Networks: A basic, vanilla GNN is only *permutation-equivariant* at the node level and *permutation-invariant* at the graph level. Assuming a canonical graph neural network as introduced by Gilmer et al., 2017, we can state graph-wise permutation-invariance by construction P as the permutation matrix from a symmetric group, and GNN-function f:

$$f(PX,\, PAP^{\top}) =\, f(X, A), \tag{5}$$

and permutation-equivariance for a node-wise function F:

$$F(PX,\, PAP^{\top}) = PF(X, A). \tag{6}$$

We can state this due to two properties: (1) permutation matrix P is orthogonal and only rearranges entries, and (2) message-passing functions utilize a permutation-invariant operator $\bigoplus$ at every update function, such that the reordering of neighbors doesn't change the result [Bronstein et al., 2021]. Thus, we can state node-wise permutation-equivariance in basic graph neural networks.

Computational chemists, biologists, and atomistic scientists desire to induce geometric priors consistent with the inherent symmetries and physics of molecular systems. For example, the energy of an atomistic system should be invariant to rotations, inversions, or translations of the system. Forces, however, should be equivariant to rotations, but invariant to translations. As such, researchers design variations of graph neural networks to induce these geometric priors, based on the intended application-space of the model.

Invariant GNNs: Using relative interatomic distances to introduce rotation and translation invariance. Within the message function $\psi(\mathbf{h_u}^{(l)}, \mathbf{h_v}^{(l)}, \mathbf{e_{uv}^{(l)}})$, we can implement various methods to induce rotation and translation invariance when predicting potential energy surface values. While many methods exist, we focus on outlining how relative distances and angles can incorporated to induce invariance.

To introduce rotational and translational invariance, one can implement interatomic distances such as

$$r_{uv} = ||x_u - x_v||_2 \tag{7}$$

where r_{uv} is embedded in $\mathbf{e_{uv}^{(l)}}$ and later passed into message function ψ. Within SchNet, Schütt et al., 2017 adopts this method to obtain rotationally invariant energy predictions. Via an energy-conserving force model obeying $F = -\nabla E$, differentiation permits rotationally equivariant forces.

Various models seek to implement additional interatomic information, such as angles between a node and neighobring embeddings. As demonstrated in DimeNet [Gasteiger et al., 2022], one can include an angle embedding within the message passing function:

$$m_{ij}^{(l+1)} = \phi_{update}(m_{ij}^{(l)}, \sum_{k \in \mathcal{N}_i \setminus j} \phi_{int}(m_{ki}, e^{(ij)}, a^{(ijk)})) \tag{8}$$

where ϕ_{update} and ϕ_{int} are MLPs, m_{ij} is message embedding between node i and j, $e^{(ij)}$ is the edge embedding between node i and j, and $a^{(ijk)}$ are the angles between nodes k and i, as well as the angles between nodes i and j. Gasteiger et al., 2022 uses radial basis functions to represent $e^{(ij)}$and spherical Bessel functions to represent $a^{(ijk)}$. From these two examples, we see how different models can modify the edge embeddings, as well as the message passing function to induce rotation and translation invariance with respect to energy value predictions. We also note how these methods of inducing invariance lie within and upstream of message

passing functions.

Equivariant GNNs: Within the application space of molecular and chemical simulation, most GNNs aim to predict both equivariant values (forces) and invariant values (total energy). With most systems using relative distances and inducing translation invariance, the techniques and methodology in this section focus on inducing rotation equivariance within models.

Scalarization Methods: Through our review of invariant GNN and GNN systems, we've seen how displacement vectors $\in \mathbb{R}^3$ are turned into invariant scalars, allowing the GNN to perform invariant computations. We can apply similar methods of scalarization but with the intention of inducing rotational equivariance within the architecture instead.

Through the work of E(n)-Equivariant Graph neural Networks (EGNNs) [Satorras et al., 2022], we see the usage of scalarization within the edge computation:

$$m_{ij} = \phi_{edge}(h_i^{(l)}, h_j^{(l)}, \|x_i^l - x_j^l\|^2, e^{(ij)}) \tag{9}$$

through the use of the relative squared distance between position vectors of node i and j, this edge embedding will always be E(n) invariant. Within the EGNN, the following step, which takes this E(n) invariant edge embedding as input, induced E(n) equivariance:

$$\vec{m}_i = (1/(M-1)) \sum_{j \neq i} (x_i^l - x_j^l) \phi_x(m_{ij}) \tag{10}$$

In this step, an orthogonal rotation and translation of x^l will follow E(n)-Equivariance. Later steps in this architecture only update invariant embeddings m_{ij} and $h^{(l)}$. Thus, with each layer, x^{l+1} is equivariant to x^l. This allows EGNN to output equivariant final positions and provide invariant outputs for molecule properties.

Higher-Degree Steerable Features: Despite scalarization methods as used by Satorras et al., 2022 being both computationally cheap and effective in implementing equivariance, we seek to utilize high-degree representations to better represent complex geometic information and to retain expressivity [Cen et al., 2025]. To develop such models, we illustrate methods of higher-degree steerable features utilized to induce equivariance.

Higher-order steerable features refer to feature representations transforming according to higher-degree ($l > 0$) irreducible representations, satisfying the condition of linear steerability:

$$\Phi(\pi(g)f) = \pi'(g)\,\Phi(f), \quad \forall g \in G \tag{11}$$

where G is a Lie group, $f \in \mathcal{F}$ denotes the input feature representation, π and π' are group representations acting on the input and output feature spaces respectively, and $\Phi : \mathcal{F} \to \mathcal{F}'$ is a linear operator such as a convolutional layer or linear map. This condition ensures that applying a group transformation to the input results in a predictable, data-independent linear transformation of the output features [Cohen et al., 2016].

- **Wigner-D matrices:** Cartesian tensors form reducible representations of the rotation group $SO(3)$. We can decompose these tensors into type-l spherical tensors. These are irreducible representations of $SO(3)$ transformations, allowing higher-order equivariant feature transformations. Type-l spherical tensors have $2l{+}1$ dimensionality where l is an integer, such that a type-0 tensor is a scalar, a type-1 tensor is a 3-dimensional vector, and thus forth. Having introduced spherical tensors, we can introduce Wigner-D matrices, defined as:

  $$\mathbf{D}_l(g) \in \mathbb{R}^{(2l+1)\times(2l+1)} \tag{12}$$

  where $g \in SO(3)$, and $\mathbf{D}_l(g)$ is the l-th degree Wigner-D matrix which describes how type-l spherical tensors transform via rotation [Gilmore, 2008]. Through the use of Wigner-D matrices and spherical tensors, we introduce a methodology that allows cartesian tensor features to exist as irreducible representations within the rotations of SO(3), simplifying the goal of achieving equivariance.

- **Spherical Harmonics:** Spherical harmonics allow us to map unit directions of vectors in $\mathbb{R}^3$ into type-l vectors. Spherical harmonics take a unit vector $\vec{x}$ on unit sphere (S^2), such that:

$$Y^{(l)}(\vec{x}) : S^2 \to \mathbb{R}^{2l+1}. \tag{13}$$

Given how spherical harmonics assume the usage of a unit vector, we can instead take vector $\vec{x}$ of $\mathbb{R}^3$ and decompose it into a euclidean norm $\|\vec{x}\|_2$ and directional unit vector $\hat{x} = \vec{x}/\|\vec{x}\|_2$. Through the language of Wigner-D matrices, we can also illustrate how spherical harmonics transform equivariantly:

$$\mathbf{Y}^{(l)}(R_g\,\hat{x}) = \mathbf{D}^{(l)}(g)\mathbf{Y}^{(l)}(\hat{x}), \quad g \in SO(3), \tag{14}$$

where we have rotation matrix $R_g \in \mathbb{R}^{3\times3}$, and l-degree Wigner-D matrix $D^{(l)}(g)$ [Han et al., 2025]. Spherical harmonics provide an inherently equivariant method to encode geometric information regarding direction. The high-degree of spherical harmonics permits additional geometric information to be captured, all while maintaining equivariance.

- **Tensor Products:** Given how spherical harmonics equip us with a basis function to represent 3D geometric features in type-l vectors, and how spherical tensors allow us to store features in irreducible representations which guarantee equivariance, we now seek a methodology that allows us to combine type-l vectors of different dimensionality, all while retaining symmetry and equivariance. To do so, we must utilize the tensor product and Clebsch-Gordon coefficients. Starting with multi-channel features $F^{(\vec{l}_1)} \in \mathbb{R}^{(2l_1+1)\times C_1}$ and $F^{(\vec{l}_2)} \in \mathbb{R}^{(2l_2+1)\times C_2}$ We can define the parametric Clebsch-Gordon tensor product as:

$$[F^{(\vec{l}_1)} \otimes_{cg}^{W} F^{(\vec{l}_2)}]^{(l)}_{m,c} = \sum_{\substack{c_1=1\\c_2=1}}^{C_1,C_2} w_{c_1c_2c} \sum_{\substack{m_1=-l_1\\m_2=-l_2}}^{l_1,l_2} V^{(l,m)}_{(l_1,m_1)(l_2,m_2)}\, \mathbf{f}^{(l_1)}_{m_1,c_1}\, \mathbf{f}^{(l_2)}_{m_2,c_2}. \tag{15}$$

where $\mathbf{f}^{(l)}_{m_1,c_1}$ indicates order m and channel c of $F^{(\vec{l})}$ for $l = l_1, l_2$, $V^{(l,m)}_{(l_1,m_1)(l_2,m_2)}$ are Clebsch-Gordan coefficients, and $w_{c_1c_2c}$ are learnable parameters [Han et al., 2025, An et al., 2025]. Through the tensor product using Clebsch-Gordan coefficients, we are able to obtain an SO(3)-equivariant product. Given Clebsch-Gordan tensor product $\otimes_{cg}^{W} : F^{(\vec{l}_1)} \times F^{(\vec{l}_2)} \to F^{(\vec{l})}$, we can use SO(3)-equivariant Wigner-D matrices to show:

$$\mathbf{D}^{(l)}(g)(\tilde{\mathbf{F}}^{(l_1)} \otimes_{\text{cg}}^{W} \tilde{\mathbf{F}}^{(l_2)}) = \left[(\mathbf{D}^{(l_1)}(g)\tilde{\mathbf{F}}^{(l_1)} \otimes_{\text{cg}}^{W} \mathbf{D}^{(l_2)}(g)\tilde{\mathbf{F}}^{(l_2)})\right]^{(l)}. \tag{16}$$

Through tensor products using Clebsch-Gordon coefficients, we obtain a methodology that allows us to combine spherical tensors of different degrees and retain equivariance.

Having learned the methodology of higher-degree steerable features, we can understand how these tools are implemented in modern GNNs to induce equivariance. To do so, we can focus on building off concepts introduced by the Tensor Field Network (TFN) Thomas et al., 2018. Using a rephrasing off the message passing steps and feature update found in TFN as introduced by Han et al., 2025, we introduce the message computation step:

$$\vec{\mathbf{M}}^{(\mathcal{L})}_{uv} = \mathbf{Y}^{(\mathcal{L})}\left(\frac{\vec{x}_{uv}}{\|\vec{x}_{uv}\|}\right) \otimes_{\text{cg}}^{W} \vec{\mathbf{V}}^{(\mathcal{L})}_{v}, \quad l \in \mathcal{L} \tag{17}$$

where we have relative positions $\vec{x_{uv}} = \vec{x_u} - \vec{x_v}$, spherical harmonic function $Y^{(\mathcal{L})}$, Clebsch-Gordan Tensor product $\otimes_{cg}^{W}$ with steerable features $\vec{\mathbf{V}}^{(\mathcal{L})}_{v}$. Importantly, $\vec{\mathbf{M}}^{(\mathcal{L})}_{uv}$ is equivariant. On the left side of the Clebsch-Gordan tensor product, we have $\mathbf{Y}^{(\mathcal{L})}\left(\frac{\vec{x}_{uv}}{\|\vec{x}_{uv}\|}\right)$, which is the spherical harmonic produced with the radial directions from v to u. On the right side of the tensor product, we have $\vec{\mathbf{V}}^{(\mathcal{L})}_{v}$ which consists of spherical tensors representing the neighboring node v's features. These are then combined via the Clesbh-Gordon coefficients, with the specification of learnable scalar weights W which are computed via an MLP f that takes distance $\|\vec{x_{uv}}\|$ such that $W = f(\|\vec{x}_{uv}\|)$. As demonstrated above, each and every one of these steps

maintains equivariance, as well as operating using irreducible representations of SO(3) transformations. This then leads to node-wise aggregation and self-interaction:

$$\vec{\mathbf{V}}_u^{(l)} = \left(\vec{\mathbf{V}}_u^{(l)} + \sum_{v \in \mathcal{N}(u)} \vec{\mathbf{M}}_{uv}^{(l)} \right) \mathbf{W}^{(l)}, \quad l \in \mathcal{L}, \tag{18}$$

where $W^{(l)}$ is a parameter matrix of $\mathbb{R}^{c_l \times c_l}$. As we utilize summation of equivariant type-l spherical tensors, we maintain equivariance. Also, as $W^(l)$ operates only on the channels. We then end using normalization and nonlinearity steps which preserve equivariance:

$$\vec{\mathbf{V}}'^{(\mathcal{L})} = \left\{ \vec{\mathbf{V}}^{(l)} \, \sigma\left(\left\| \vec{\mathbf{V}}^{(l)} \right\|_2 + b^{(l)} \right) \right\}_{l \in \mathcal{L}}, \tag{19}$$

with activation function σ and L2 normalization. Thus, we demonstrate how we can utilize high-steerable features to induce equivariance in a model. Modern GNNs which implement equivariance improve upon this framework through varying activation functions, multi-layer perceptrons at the update function, radial Bessel functions, higher body interactions, and more.

3 From GNNs to Transformers, Methods of Equivariance

Having reviewed the basic methods of inducing equivariance in Graph Neural Networks, we now move on to recognizing the connection between Graph Neural Networks and Transformer architectures. Joshi, 2025 demonstrates this connection in architectures thoroughly in his article "Transformers are Graph Neural Networks". To avoid redundancy, we assume knowledge from his article, as well as familiarity with Transformer architecture. Instead, we seek to respond to this work, seeking to answer whether we could, using the proof of connection between a graph neural network and Transformer, create an equivariant Transformer.

3.1 Bridging GNNs to Transformers via Attention and Complete Graphs

For an in-depth illustration of the connection between Graph Neural Networks to Graph Attention Networks, and so forth to Transformers, please see Joshi, 2025's "Transformers are Graph Neural Networks" technical article. We briefly recap the logical reasoning and connections between the architectures. As described in Section 2.1, we can summarize the key functions of GNNs to be:

$$h_u^{(l+1)} = \phi\left(\mathbf{h}_u^{(l)}, \bigoplus_{v \in \mathcal{N}_u} m_{uv}^{(l)} \right) \tag{20}$$

where:

$$m_{uv}^{(l)} = \psi(\mathbf{h_u}^{(l)}, \mathbf{h_v}^{(l)}, \mathbf{e_{uv}^{(l)}}), \tag{21}$$

with $\mathbf{h_u}^{(l)}$ being the hidden node embedding for node u at layer l, $e_{uv}^{(l)}$ being the edge embedding between nodes u and v, and ψ and ϕ being learnable functions.

To move towards Graph Attention Networks (GATs), we introduce the concept of attention. Simply put, attention allows us to weight the importance of features between two nodes.Veličković et al., 2018 introduces attention coefficients:

$$e_{uv} = a(\mathbf{W}\vec{h_u}, \mathbf{W}\vec{h_v}), \tag{22}$$

and coefficient normalization:

$$\alpha_{uv} = \text{softmax}_v(e_{uv}) = \frac{\exp(e_{uv})}{\sum_{k \in \mathcal{N}_u} \exp(e_{uk})}. \tag{23}$$

We have weight matrix $\mathbf{W} \in \mathbb{R}^{F' \times F}$ applied to each node h, and $k \in \mathcal{N}_u$ as the node indicies that are in the neghborhood of u [Veličković et al., 2018]. In the original GAT paper, they introduce the attention mechanism $a : \mathbb{R}^{F'} \times \mathbb{R}^{F'} \to \mathbb{R}$ as a single-layer neural network with weight vector $\vec{a} \in \mathbb{R}^{2F'}$. As such, they

write $e_{uv} = \text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_u \,\|\, \mathbf{W}\mathbf{h}_v])$, where they first concatenate the weighted node embeddings, apply the weight vector, and then apply LeakyReLU nonlinearity. To maintain consistency with attention found in Transformers, we can use the $W_Q, W_K, W_V \in \mathbb{R}^{F \times F}$ scoring method of attention so that we can state [Vaswani et al., 2017]:

$$e_{uv} = (W_Q h_u)^\top (W_K h_v) \tag{24}$$

and normalizing:

$$\alpha_{uv} = \text{softmax}_v\big((W_Q h_u)^\top (W_K h_v)\big) = \frac{\exp\big((W_Q h_u)^\top (W_K h_v)\big)}{\sum_{k \in \mathcal{N}_u} \exp((W_Q h_u)^\top (W_K h_k))}\,. \tag{25}$$

.

Having obtained our attention coefficients, we can now reason how to incorporate these into message-passing functions to update node embeddings h. Following the methodology of Veličković et al., 2018, we can represent the message-passing function as:

$$\vec{m_{uv}} = \psi(\vec{h_u^l}, \vec{h_v^l}) = \alpha_{uv} \cdot W_V \vec{h_v}, \tag{26}$$

. Now that we have determined how the message-passing function utilizes attention weights, we now direct ourselves towards using these embeddings to update the node embeddings. Similarly to GNNs, we are likely to implement steps of summation and nonlinearity. However, we can implement multi-head attention to capture various representations. Each head allows it's own calculation of attention weights and corresponding $W_Q^{(k)}, W_Q^{(k)}, W_Q^{(k)}$ embeddings. In GATs, multi-head attention typically involves concatenating K heads into an intermediate node embedding of size $\mathbb{R}^{KF}$, which is projected into this dimension for each hidden layer until the final layer, which averages K heads as shown here:

$$\vec{\mathbf{h}}'_u = \sigma\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{j \in \mathcal{N}_u} \vec{m_{uj}^{(k)}}\right) \tag{27}$$

with a residual connection to create our update node embedding:

$$\vec{h_u^{l+1}} = \vec{h_u'} + \vec{h_u^l} \tag{28}$$

where k refers to the k-th attention head, specifically $\alpha_{uv}^k W_V^k$ specific to the k-th attention head, and nonlinearity function σ. Now that we have successfully outline how we now use attention to produce our message passing step using node embeddings, we must also emphasize how we use masked attention in the sense we only compute e_{uv} for $v \in \mathcal{N}_u$ where $\mathcal{N}_u$ is the set of nodes in the neighborhood of node u. In the application of material simulation, a cutoff radius usually specifies which nodes are connected or not. Having done the heavy-lifting of introducing attention mechanisms in GNNS to produce GATs, we can now easily demonstrate how to go from GATs to Transformers.

In the simplest explanation, Transformers are GATs where every and all nodes are connected, forming a complete graph. In addition to this, self-attention is computed between all nodes, moving from local-attention to global-attention. We can declare set $\mathcal{V}$ as the set including all nodes. Thus, when computing the attention coefficient and message embeddings, we now set the neighborhood condition to be $j \in \mathcal{V}$. The Transformer architecture works with multi-head attention with a slight difference. Whereas the GAT architecture concatenates intermediate node embeddings and averages the embeddings by heads for the final layer, Transformers concatenate multi-head message vectors and apply a learnable matrix W_O, such as:

$$m_u^l = \big[\,\mathbf{z}_u^{(1)} \,\|\, \mathbf{z}_u^{(2)} \,\|\, \cdots \,\|\, \mathbf{z}_u^{(K)}\,\big]\mathbf{W}_O, \tag{29}$$

where each head output is given by

$$\mathbf{z}_u^{(k)} = \sum_{v \in \mathcal{V}} \vec{m_{uv}^{(k)}}\,. \tag{30}$$

Whereas GATs 'mix' inputs across KF channels using $W_V^{(k,l+1)}$ for downstream intermediate layers, Transformers use the matrix W_O to mix information from various heads and return the matrix to the original

model size within the same layer. We can now take this final message embedding of $m_u^l \in \mathbb{R}^F$, and apply a residual connection to get our final node output:

$$h_u^{l+1} = \phi(\text{LayerNorm}(h_u^l + m_u^l)), \tag{31}$$

where we set ϕ to be a feedforward network instead. In practice, the multi-head attention mechanism can be applied similarly in either way for both GATs or Transformers. In addition, the final layer may include additional LayerNorm and MLPs. More importantly, however, we construct how simply designating a GAT as a fully-connected graph allows the architecture to effectively become a Transformer.

3.2 Inducing Equivariance in a Transformer:

Having derived a Transformer architecture starting from a GNN, we see how the key difference lies in the message passing steps whose weights come only from the attention mechanism. We hope to use our intuition from inducing equivariance in GNNs, and use similar reasoning used to derive Transformers to ideate equivariant Transformers.

Vanilla Transformer: Before we assess methods of inducing equivariance, we seek to demonstrate why the canonical, bare-bones Transformer architecture fails to satisfy equivariance. For a Transformer to be equivariant, all steps throughout the model must obey rules of equivariance as outlined in Equation 1. For this property to be retained, we must use sub-operations that are either invariant or equivariant on the condition that the composition of these operations retains equivariance, such as an invariant scalar multiplying an equivariant operation. Through analyzing the message passing function [Eq. 26], we notice how the message passing function $\vec{m_{uv}}$ can be the initial and fundamental failure point towards exhibiting equivariance. Repeating Equation 26:

$$\vec{m_{uv}} = \psi(\vec{h_u^l}, \vec{h_v^l}) = \alpha_{uv} \cdot W_V \vec{h_v}, \tag{26}$$

for this message passing function to be equivariant, we need all and every sub-operation to satisfy the properties. Focusing on the input vector $\vec{h_v}$, we must be able to illustrate that $\vec{h_v}$ behaves either invariantly or equivariantly given a group representation. If h_v is a scalar (type-0), we obtain invariance. If $\vec{h_v} \in \mathbb{R}^3$ such that $\vec{h_v} \mapsto R\vec{h_v}, \quad R \in SO(3)$. Moving on to the learnable matrix W_v, this matrix will not necessarily exhibit equivariance. Assuming all other parts are equivariant/invariant, the matrix will only satisfy the desired condition if $W_v = \lambda I$, where λ is any scalar. Finally, we attend to the attention weight. Assuming the rest of the function satisfies equivariance, we need our scalar attention weight to satisfy invariance as it is a scalar:

$$\alpha_{uv}\big(D(g)h_u,\ D(g)h_v\big) = \alpha_{uv}(h_u, h_v) \qquad \forall g \in G. \tag{32}$$

where $D(g)$ is the group representation from Lie Group G. If we analyze our scoring function from Equation 24, we repeat:

$$e_{uv} = (W_Q h_u)^\top (W_K h_v) \tag{24}$$

Focusing on the scoring function, we arrive at the two issues we've already encountered, but with the specification that we need invariance. For invariance, we need weight vectors W_K and W_Q to obey Schur's lemma, and for input h_v to be a scalar. Through focusing on the message-passing function, we identify how the computation of the scoring function and input vectors pose likely issues for trivially inducing equivariance. While later operations in the Transformer architecture can possibly remove equivariance, choosing representation-aware nonlinearity functions, normalization layers, and more allow geometric priors to persist.

Equivariance via Scalarization: Similar to how scalarization constructed invariant representations that preserved equivariance when passed through equivariant operations downstream for GNNs, we can implement similar techniques in order to do the same for Transformers. At its core, we hope to use only relative distances, as well use only invariant representations h_i for each node. We also focus on creating attention scores that are computed only from invariant quantities. This, therefore, permits the attention weight to be invariant in respect to the input itself. Thus, we enforce input features to be scalar $h_u \in \mathbb{R}^1$. Given the group representation $D(g) \quad \forall g \in G$, we can state:

$$(W_Q D(g) h_u^{(0)})^\top (W_K D(g) h_v^{(0)}) = (W_Q h_u^{(0)})^\top (W_K h_v^{(0)}). \tag{33}$$

This also permits matrices W_Q and W_K to be learnable, as learnable matrices projecting on scalars will retain invariance. Using invariant quantities when producing the attention matrix allows for the message passing function to largely stay invariant. Through utilizing equivariance-retaining normalization and non-linearity functions, one can construct a Transformer obeying equivariance. TorchMD-Net is an example of an equivariant Transformer that uses relative distances and involves equivariance-retaining steps throughout the architecture, achieving E(3)-equivariance [**thlke2022torchmdnetequivariantTransformersneural**].

Equivariance via High-Degree Steerable Features: As we've explored regarding GNNs, there is a strong interest in working with tensors with degree $l > 1$, especially in regards to model expressivity. Luckily, we can extend many concepts and methods used in constructing high-degree steerable equivariant GNNs for Transformers as well.

To begin construction of a Transformer using such a representation, we must first establish how all features must be irreducible representations in the form of equivariant tensors, angular information can be encoded in spherical harmonics, and how we hope to use the Clebsch-Gordon tensor product to retain equivariance. In regards to the attention coefficients, we can maintain the same strategy as used in "Equivariance via Scalarization" and seek to produce scalar invariant attention coefficients. Through following this chain of reasoning, we can guide the derivation of an equivariant high-degree feature Transformer. We also note that there are variations to how to do this, and as such, we aim to stay consistent with basic principles outlined by Fuchs et al., 2020. To begin, we can reference the attention coefficient mechanism:

$$e_{uv} = (W_Q h_u)^\top (W_K h_v) \tag{24}$$

.

Once again, we seek a method of which to take input vectors h_u and h_v and learnable matrices W_Q and W_K and transform them into forms which preserve equivariance. We can focus on making $(W_Q h_u)$ and $(W_K h_v)$ rotationally equivariant, as their dot product will yield a rotationally invariant outcome. First, we can first work with forms of h_u and h_v that are of irreducible representations. For nodes u and v, we can use embeddings:

$$\mathbf{V}_u^{(\mathcal{L})} \in \mathcal{V}(\mathcal{L}), \quad \mathbf{V}_v^{(\mathcal{L})} \in \mathcal{V}(\mathcal{L}), \tag{34}$$

where

$$\mathcal{V}(\mathcal{L}) := \bigoplus_{\ell \in \mathcal{L}} \mathbb{R}^{2\ell+1} \otimes \mathbb{R}^{C_\ell}, \tag{35}$$

demonstrating the use of spherical tensors that transform according to the irreducible representations of $SO(3)$, where $\bigoplus$ indicated direct summation. The next step is to transform our weighted query embedding to be equivariant. Thankfully, this is relatively simple, as W_Q acts on the channels of $\mathbf{V}^{(\mathcal{L})}$ and therefore maintains equivariance via properties of the spherical tensors, such that we can write:

$$Q_u^{(\mathcal{L})} = V_u^{(\mathcal{L})} W_Q, \tag{36}$$

such that $Q_u^{(\mathcal{L})}$ represent the equivariant query embedding component bundled for all degrees L. Moving onto the key component, we wish to further incorporate geometric information regarding the node and it's neighbor node, similar to as done in TFN [Thomas et al., 2018]. We can model this section closely to the message computation step as illustrated in Equation 17. We can use a spherical harmonic to encode the angular information, utilize the spherical tensor form of the neighboring node, and apply the Clebsch-Gordon tensor product. As such, we can write:

$$K_{uv}^{(\mathcal{L})} = \mathbf{Y}^{(\mathcal{L})}\left(\frac{\vec{x}_{uv}}{\|\vec{x}_{uv}\|}\right) \otimes_{\text{cg}}^{W_K} \vec{\mathbf{V}}_v^{(\mathcal{L})}, \tag{37}$$

by abiding by all rules of equivariance as mentioned along with Eq. 17. We note that matrix W_K can be learnable or through an MLP based of interatomic distances, depending on the method. Now that we are equipped with equivariant query and key embeddings, we can rewrite our attention coefficients:

$$e_{uv} = (Q_u^{(\mathcal{L})})^\top (K_{uv}), \tag{38}$$

which obeys rotational equivariance in components:

$$Q_u \mapsto Q'_u = D(g)Q_u, \qquad K_{uv} \mapsto K'_{uv} = D(g)\, K_{uv}, \qquad g \in SO(3) \tag{39}$$

and demonstrates rotational invariance:

$$e'_{uv} = Q'^{\top}_u K'_{uv} = Q^{\top}_u D(g)^{\top} D(g)\, K_{uv} = Q^{\top}_u K_{uv} = e_{uv}, \tag{40}$$

where $D(g)$ are group representation that are orthogonal, such that $D(g)^{\top}D(g) = I$. Having proved that we can obtain rotationally invariant values for e_{uv}, we can follow Eq. 23, and obtain α_{uv}. Referencing the intuition on focusing on constructing an equivariant message passing function as in Eq. 26, we realize we have obtained rotationally invariant values α_{uv}, suggesting we must now obtain and equivariant form of the weighted value embedding in order to obtain a fully rotationally equivariant message-passing function. Following the logic of the SE(3)-Transformer, we seek to implement geometric information in the value step. As such, we can state:

$$V^{(\mathcal{L})}_{uv} = \mathbf{Y}^{(\mathcal{L})}\left(\frac{\vec{x}_{uv}}{\|\vec{x}_{uv}\|}\right) \otimes^{W_V}_{\text{cg}} \vec{\mathbf{V}}^{(\mathcal{L})}_{v}, \tag{41}$$

which, matching reasoning with Eq. 37, we can reason how $V^{(\mathcal{L})}_{uv}$ is rotationally equivariant. Obtaining this component, we can craft a rotationally equivariant form of Eq. 26:

$$M_{uv} = \alpha_{uv}\, V^{(\mathcal{L})}_{uv}, \tag{42}$$

such that m_{uv} is SE(3) equivariant. We can now follow the rest of the steps to construct our , in specific, updating the node features using self-interaction:

$$V'^{(\mathcal{L})}_{u} = W_{self} V^{(\mathcal{L})}_{uv} + \sum_{v \in \mathcal{N}_{\sqcap}} M_{uv}, \tag{43}$$

where W_{self} implement self-interaction. We now have demonstrated the basic path to achieve equivariance in a architecture. We focus on producing an invariant attention weight, and constructing an equivariant message between such nodes. The SE(3)-Transformer implements equivariance similarly to this methods with some variations in the computation of weights, as well as in regards to obtaining the final feature outputs [Fuchs et al., 2020].

4 Discussion

Through this work, we explore how to induce geometric priors in both graph neural networks and through the intuition built by understanding the connection between GNN and architectures. We also understand the importance of achieving equivariance in the message-passing function, as well as obtaining an invariant attention coefficient. All in all, this paper intends to guide an intuition towards developing equivariant architectures that can incorporate attention.

Despite our review yielding ideas and connections regarding geometric priors in attention-based architectures, we still question and investigate the need for geometric priors. Kreiman et al., 2025 demonstrates the use of a standard Transformer architecture for interatomic potential use, achieving not only competitive accuracy with state-of-the-art equivariant GNNs, but also near perfect coefficient of determination in regards to equivariant similarity on the OMol25 dataset. This may suggest that researchers should opt away from constructing models with geometric priors, or they should focus more on constructing large and diverse datasets such that a geometrically-unconstrained model can learn equivariance Qu et al., 2024. Nonetheless, we hope to investigate scenarios where such actions or conditions are not applicable, such as a desire to achieve transformer-like parallelization and speedup in a model whilst working off a smaller dataset. In future works, we hope to validate this problem areas, as well as reason and demonstrate the possible increase in efficiency and accuracy through attention-based architectures with geometric priors.

5 Acknowledgements

This paper is a rough draft with plenty of typos, mathematical errors, and more. As an earlier reader of this draft, the authors request any and all suggestions for improvement. This paper was also written by humans without explicit writing by generative AI, and the authors accept all errors as their own.

References

An, Junyi et al. (2025). *Equivariant Spherical Transformer for Efficient Molecular Modeling*. arXiv: 2505.23086 [cs.LG]. URL: https://arxiv.org/abs/2505.23086.

Battaglia, Peter et al. (2018). *Relational inductive biases, deep learning, and graph networks*. URL: https://arxiv.org/pdf/1806.01261.pdf.

Bronstein, Michael M. et al. (2021). *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. arXiv: 2104.13478 [cs.LG]. URL: https://arxiv.org/abs/2104.13478.

Cen, Jiacheng et al. (2025). *Are High-Degree Representations Really Unnecessary in Equivariant Graph Neural Networks?* arXiv: 2410.11443 [cs.LG]. URL: https://arxiv.org/abs/2410.11443.

Cohen, Taco S. and Max Welling (2016). *Steerable CNNs*. arXiv: 1612.08498 [cs.LG]. URL: https://arxiv.org/abs/1612.08498.

Fuchs, Fabian B. et al. (2020). *SE(3)-Transformers: 3D Roto-Translation Equivariant Attention Networks*. arXiv: 2006.10503 [cs.LG]. URL: https://arxiv.org/abs/2006.10503.

Gasteiger, Johannes, Janek Groß, and Stephan Günnemann (2022). *Directional Message Passing for Molecular Graphs*. arXiv: 2003.03123 [cs.LG]. URL: https://arxiv.org/abs/2003.03123.

Gilmer, Justin et al. (2017). *Neural Message Passing for Quantum Chemistry*. arXiv: 1704.01212 [cs.LG]. URL: https://arxiv.org/abs/1704.01212.

Gilmore, Robert (2008). *Lie Groups, Physics, and Geometry: An Introduction for Physicists, Engineers and Chemists*. Cambridge University Press.

Han, Jiaqi et al. (2025). *A Survey of Geometric Graph Neural Networks: Data Structures, Models and Applications*. arXiv: 2403.00485 [cs.LG]. URL: https://arxiv.org/abs/2403.00485.

Joshi, Chaitanya K. (2025). *Transformers are Graph Neural Networks*. arXiv: 2506.22084 [cs.LG]. URL: https://arxiv.org/abs/2506.22084.

Kreiman, Tobias et al. (2025). *Transformers Discover Molecular Structure Without Graph Priors*. arXiv: 2510.02259 [cs.LG]. URL: https://arxiv.org/abs/2510.02259.

Qu, Eric and Aditi S. Krishnapriyan (2024). *The Importance of Being Scalable: Improving the Speed and Accuracy of Neural Network Interatomic Potentials Across Chemical Domains*. arXiv: 2410.24169 [cs.LG]. URL: https://arxiv.org/abs/2410.24169.

Satorras, Victor Garcia, Emiel Hoogeboom, and Max Welling (2022). *E(n) Equivariant Graph Neural Networks*. arXiv: 2102.09844 [cs.LG]. URL: https://arxiv.org/aAbs/2102.09844.

Schütt, Kristof T. et al. (2017). *SchNet: A continuous-filter convolutional neural network for modeling quantum interactions*. arXiv: 1706.08566 [stat.ML]. URL: https://arxiv.org/abs/1706.08566.

Thomas, Nathaniel et al. (2018). *Tensor field networks: Rotation- and translation-equivariant neural networks for 3D point clouds*. arXiv: 1802.08219 [cs.LG]. URL: https://arxiv.org/abs/1802.08219.

Vaswani, Ashish et al. (2017). "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Veličković, Petar et al. (2018). *Graph Attention Networks*. arXiv: 1710.10903 [stat.ML]. URL: https://arxiv.org/abs/1710.10903.